

Unicon Unit Testing Framework User's Guide

Ian Trudel

Unicon Technical Report: 23

2024-05-08

Abstract

This technical report introduces a unit testing framework for Unicon, inspired from Kent Beck's SUnit [1]. The framework aims to provide a robust testing environment for Unicon applications, simplifying the process of writing and executing tests.

Unicon Project

<http://unicon.org>

University of Idaho

Department of Computer Science
Moscow, ID, 83844, USA

1 Introduction

Unit testing is a critical part of the development process, ensuring that individual units of source code function correctly. This new framework brings structured and repeatable testing to Unicon programs and libraries whilst leveraging Unicon's Object Oriented Programming features [9].

2 Components of the Framework

2.1 TestSuite

A test suite allows to aggregate multiple test cases in order to facilitate their collective execution.

```
—
“add(testCase)” — adding a test case
add(testCase) adds a TestCase instance to the suite.
—
“run()” — run a test suite
run() executes all tests in a suite and returns an exit status.
```

2.2 TestReporter

`TestReporter` collects and displays test results in a readable format. A `TestReporter` can be subclassed and passed onto a `TestSuite` for a custom reporting format.

```
—
“result(cname, mname, passed)” — adding a result
result(cname, mname, passed) adds the result of a test including the
classname, method name and whether it passed or not.
—
“results()” — returns the test results
results() returns the test results including the number of tests that have
passed, failed and the total runtime, as a record total(passed, failed, runtime).
—
“runtime(time)” — sets the total runtime
runtime(time) sets the total runtime for the entire TestSuite.
—
```

—
“summary()” — display the tests summary

`summary()` reports the test results including the number of tests that have passed, failed and the total runtime in a human-readable format. It also returns a record `total(passed, failed, runtime)`.

2.3 TestCase

`TestCase` serves as the base class for all tests. It is necessary to subclass `TestCase` in order to write individual tests. It provides methods for set up and tear down the test environment both on a per-class and per-test basis. Every method starting with `test` in the subclass will be automatically recognized and run by the test suite.

—
“setupClass()” — setting up a class

`setupClass()` initializes resources before any tests in the class run. It is used to allocate or initialize resources shared across all tests in a test class, ensuring they are available before any tests are executed.

—
“teardownClass()” — tearing down a class

`teardownClass()` cleans up resources after all tests in the class have run. It ensures that all allocated resources in `setupClass()` are properly released or cleaned up after all tests have completed.

—
“setup()” — setting up a test

`setup()` initializes resources before each individual test runs, ensuring a consistent starting point for all tests.

—
“teardown()” — tearing down a test

`teardown()` cleans up resources after each test individual test runs, maintaining isolation between tests.

2.4 Assertions

“assertEqual(expected, actual)” — assert equal values

`assertEqual(expected, actual)` verifies that the actual value generated is equal to the expected value.

—
“assertNotEqual(unexpected, actual)” — assert unequal values

`assertNotEqual(unexpected, actual)` verifies that the actual value generated is not equal to the unexpected value.

—

“assert { expression }“ — assert expression

`assert { expression }` evaluates an expression, and if true, the test will pass, otherwise will fail.

—

“assertFail { expression }“ — assert fail expression

`assertFail { expression }` evaluates an expression, and if false, the test will pass, otherwise will fail.

2.5 Mock

“expect(mname, retval)“ — set an expectation for a method call

`expect(mname, retval)` sets an expectation for the method call `mname` and returns the value `retval`.

—

“verify(mname, args)“ — verify a given method exists

`verify(mname, args)` verifies whether a method exists or not, will return the name of the method or fail.

—

“invoke(mname, args)“ — invoke a method

`invoke(mname, args)` simulates the invocation of a method `mname` with the arguments `args`. It checks whether the call matches the expectations or not and returns the expected value, otherwise will fail.

3 Example Usage

3.1 A simple Calculator

This example demonstrates the basic functionality of the unit testing framework using a simple Calculator class. The tests cover the basic addition and subtraction methods, ensuring that the calculator performs these operations correctly. This example serves as an introduction to creating test cases, setting up the test environment, and validating the outcomes using assertions.

calculator.icn

```
class Calculator()  
  method add(n, m)  
    return n + m  
  end  
  
  method sub(n, m)  
    return n - m  
  end  
end
```

test_calculator.icn

```
link calculator  
  
import unittest  
  
class TestCalculator : TestCase(calc)  
  method setup()  
    calc := Calculator()  
  end  
  
  method testAdd()  
    assertEquals(10, calc.add(6, 4))  
  end  
  
  method testSub()  
    assertEquals(3, calc.sub(8, 5))  
  end  
end  
  
procedure main()  
  ts := TestSuite()  
  ts.add(TestCalculator())  
  exit(ts.run())  
end
```

Output

```
TestCalculator_testAdd

TestCalculator_testSub

=== TEST SUMMARY ===
passed: 2, failed: 0
ran 2 tests in 0.000062s
```

Highlights

- `setup()` initializes a new instance of `Calculator()` before each test.
- `testAdd()` asserts that the add method works correctly.
- `testSub()` asserts that the sub method works correctly.
- The test summary reports the two (2) tests have passed.

3.2 Logger

The logger example demonstrates the framework's ability to handle more complex objects and interactions, such as file operations. The `Logger` class includes methods for opening a log file, writing messages, rotating logs and closing the log file. This example illustrates the use of `setup()` and `teardown()` methods to manage file system side effects, ensuring each test runs in a clean environment. The tests validate the logger's ability to handle maximum file size constraints, correctly write log messages, and perform log rotation.

`logger.icn`

```
class Logger(__filename, __filehandle, __max_size)
  method open(filename)
    self.__filename := filename
    self.__filehandle := ::open(self.__filename, "a") |
      fail("Unable to open log file")
  end

  method maxsize(size)
```

```

        return self.__max_size :=      size | self.__max_size
    end

    method write(message)
        stats := stat(self.__filename)
        if stats.size > self.__max_size then
            self.rotate()
            ::write(self.__filehandle, message || "          n")
        end

    method close()
        ::close(self.__filehandle)
    end

    method rotate()
        ::close(self.__filehandle)
        rename(self.__filename, self.__filename || ".old")
        open(self.__filename)
    end

initially
    self.__max_size := 1024
end

```

test_logger.icn

```

link logger, io

import unittest

class TestLogger : TestCase(logger, filename)
    method setup()
        filename := "logfile.log"

        logger := Logger()
        logger.open(filename)
    end

    method teardown()

```

```

        remove(filename)
        exists(filename || ".old") & remove(filename || ".old")
    end

    method testMaxSize()
        assertEquals(1024, logger.maxsize())
        assertEquals(888, logger.maxsize(888))
        assert { -1 ~= logger.maxsize(-1) }
    end

    method testWriteLog()
        s := "Welcome to Unicon"

        logger.write(s)

        file := open(filename, "r")
        contents := reads(file)
        assert { find(contents, s) > 0 }
        close(file)
    end

    method testLogRotation()
        assertEquals(10, logger.maxsize(10))

        every 1 to 20 do logger.write("rotate")

        assert { exists(filename || ".old") }
    end
end

procedure main()
    ts := TestSuite()
    ts.add(TestLogger())
    exit(ts.run())
end

```

Output

```
TestLogger_testMaxSize
```



```
Assertion failed.  
  
TestLogger_testWriteLog  
  
TestLogger_testLogRotation  
  
=== TEST SUMMARY ===  
passed: 2, failed: 1  
ran 3 tests in 0.002208s
```

An assertion fails in `testMaxSize()` because the method `maxsize()` does not handle negative size properly. Here's a revised implementation:

logger.icn

```
method maxsize(size)  
  return self.__max_size := (( size > 0) & size) | self.__max_size  
end
```

Output

```
TestLogger_testMaxSize  
  
TestLogger_testWriteLog  
  
TestLogger_testLogRotation  
  
=== TEST SUMMARY ===  
passed: 3, failed: 0  
ran 3 tests in 0.002033s
```

Highlights

- `setup()` initializes a `Logger` instance and opens a log file before each test, ensuring that each test starts with a new log file.
- `teardown()` cleans up by removing the log file and any old log file after each test, maintaining a clean test environment.

- `testMaxSize()` tests the `maxsize` method to ensure it correctly sets and returns the maximum log file size.
- `testWriteLog()` validates that the `write` method correctly writes messages to the log file.
- `testLogRotation()` ensures that log rotation is triggered when the log file exceeds the specified maximum size, and that the old log file is correctly renamed.

3.3 Fixtures

Fixtures are pre-defined data that are used as a foundation for running tests. They help ensure consistency and reliability across tests by providing a known environment. This is particularly useful when testing code that interacts with external resources such as a databases, files or network connections. Fixtures are set up before the tests are run and are torn down afterwards to clean up any changes made during testing.

fixtures_bank_account.sql

```
CREATE TABLE IF NOT EXISTS accounts (
    account TEXT PRIMARY KEY,
    holder TEXT NOT NULL,
    balance REAL DEFAULT 0.0
);

INSERT INTO accounts (account, holder, balance) VALUES
    ('123456789', 'Ada Lovelace', 10000.00),
    ('987654321', 'Alan C. Kay', 99801.28),
    ('123454321', 'Ralph Griswold', 48000.00);

CREATE TABLE IF NOT EXISTS transactions (
    transaction_id INTEGER PRIMARY KEY AUTOINCREMENT,
    account TEXT,
    type TEXT CHECK(type IN ('deposit', 'withdrawal')),
    amount REAL,
    FOREIGN KEY (account) REFERENCES accounts(account)
);
```

```
INSERT INTO transactions (account, type, amount) VALUES
  ('123456789', 'deposit', 200.00),
  ('987654321', 'withdrawal', 1500.00),
  ('123454321', 'deposit', 100.00),
  ('987654321', 'deposit', 75.00),
  ('123456789', 'withdrawal', 3456.75);
```

Use SQLite to create bank.db:

```
sqlite3 bank.db < fixtures_bank_account.sql
```

bank_account.icn

```
class BankAccount(account, holder, balance)
  method deposit(amount)
    if amount > 0 then balance += amount
  end

  method withdraw(amount)
    if 0 < amount <= balance then balance -= amount else fail("overdraft")
  end

initially
  /balance := 0.0
end
```

test_bank_account.icn

```
link bank_account

import unittest
import sqlite

record transaction(id, account, type, amount)
```

```

class TestBankAccount : TestCase(db, accounts)
  method setupClass()
    db := SQLite("bank.db")
  end

  method teardownClass()
    db.Close()
  end

  method setup()
    accounts := []
    db_accounts := db.SQL_As_List("SELECT * FROM accounts")
    every db_account := !db_accounts do {
      put(accounts, BankAccount ! db_account)
    }
  end

  method findAccount(account)
    every __account := !accounts do {
      if __account.account == account then suspend __account
    }
  end

  method testValidAccount()
    assert { 0 < *accounts }
    every account := !accounts do {
      assert { 0 < *account.account }
      assert { 0 < *account.holder }
      assert { 0.0 <= account.balance }
    }
  end

  method testTransactions()
    transactions := []
    db_transactions := db.SQL_As_List("SELECT * FROM transactions")
    every db_transaction := !db_transactions do {
      put(transactions, transaction ! db_transaction)
    }

    every transaction := !transactions do {

```

```

        account := findAccount(transaction.account)
        case transaction.type of {
            "deposit": {
                balance := account.balance
                account.deposit(transaction.amount)
                assertEquals(balance + transaction.amount, account.balance)
            }
            "withdrawal": {
                balance := account.balance
                account.withdraw(transaction.amount)
                assertEquals(balance - transaction.amount, account.balance)
            }
        }
    }
end

method testOverdraft()
    every account := !accounts do {
        balance := account.balance
        assertFail { account.withdraw(balance * 10) }
        assertEquals(balance, account.balance)
    }
end

end

procedure main()
    ts := TestSuite()
    ts.add(TestBankAccount())
    exit(ts.run())
end

```

Output

```

TestBankAccount_testValidAccount

TestBankAccount_testTransactions

TestBankAccount_testOverdraft

```

```
=== TEST SUMMARY ===  
passed: 3, failed: 0  
ran 3 tests in 0.004026s
```

Highlights

- `setUpClass()` initializes the SQLite database connection before any tests run, ensuring that the database is ready for operations.
- `tearDownClass()` closes the database connection after all tests have run, ensuring that resources are properly released.
- `setUp()` loads bank accounts from the database into the accounts list before each test, providing a fresh set of data.
- `findAccount(account)` is a utility method to locate an account by account number.
- `testValidAccount()` validates that each account has an account number, account holder and a balance.
- `testTransactions()` validates deposit and withdrawal transactions against the database.
- `testOverdraft()` tests that overdraft attempts fail correctly, ensuring no account balance goes negative.
- The test summary reports the three (3) tests have passed.

3.4 Mock objects

Test-driven development (TDD) encourages the use of mock objects as a test-first strategy. Mock objects aim to replicate the behaviour of actual objects without their actual implementation, thus allowing tests to be written nonetheless. The following example demonstrates how to implement tests for temperature sensors that are not yet implemented (or physically unavailable).

temperature_sensor.icn

```

class TemperatureSensor()
  method read()
    stop("TemperatureSensor: not yet implemented.")
  end
end

class TemperatureMonitor(sensors, threshold, callback)
  method check()
    retval := 0

    every sensor := !self.sensors do {
      temperature := sensor.read()
      if temperature > self.threshold then {
        self.callback(sensor, temperature)
        retval += 1
      }
    }

    return (retval == 0 & 1) | (retval > 0 & fail)
  end
end

```

test_temperature_sensor.icn

```

link temperature_sensor

import unittest

class MockTemperatureSensor : TemperatureSensor : Mock()
  method read()
    return self.invoke("read")
  end
end

procedure alert(sensor, temperature)
  write("Sensor " || image(sensor) || " temperature " ||
    temperature || " exceeds threshold")

```

```

end

class TestTemperatureMonitor : TestCase(monitor, sensors)
  method setup()
    self.sensors := [
      MockTemperatureSensor(),
      MockTemperatureSensor(),
      MockTemperatureSensor()
    ]

    self.monitor := TemperatureMonitor(self.sensors, 115, alert)
  end

  method testCheckTemperature()
    every sensor := !self.sensors do {
      sensor.expect("read", ?40)
    }

    assert { self.monitor.check() }
  end

  method testCheckTemperatureThreshold()
    every sensor := !self.sensors do {
      sensor.expect("read", 115 + ?100)
    }

    assertFail { self.monitor.check() }
  end
end

procedure main()
  ts := TestSuite()
  ts.add(TestTemperatureMonitor())
  exit(ts.run())
end

```

Output

```
TestTemperatureMonitor_testCheckTemperature
```



```
TestTemperatureMonitor_testCheckTemperatureThreshold
Sensor object MockTemperatureSensor_4(1) temperature 136 exceeds threshold
Sensor object MockTemperatureSensor_5(1) temperature 199 exceeds threshold
Sensor object MockTemperatureSensor_6(1) temperature 215 exceeds threshold

=== TEST SUMMARY ===
passed: 2, failed: 0
ran 2 tests in 0.000807s
```

Highlights

- `MockTemperatureSensor` class inherits from both `TemperatureSensor` and `Mock`, thus allowing to mock the `read()` method.
- `setup()` defines a series of sensors to be monitored along with a threshold temperature and a callback method.
- `testCheckTemperature()` checks that the temperature reported by the sensors are within an acceptable threshold.
- `testCheckTemperatureThreshold()` checks that the temperature reported by the sensors are not within an acceptable threshold.
- The test summary reports the two (2) tests have passed.

4 References

References

- [1] John R. Levine and Tony Mason and Doug Brown. "Lex & Yacc". O'Reilly and Associates. 1992.
- [2] K. Ray and Ray Pereda and Clinton Jeffery. "Ulex – A Lexical Analyzer Generator for Unicon". <http://unicon.org/utr/utr2/utr2a.pdf>. 2003.
- [3] Clinton Jeffery. "How to Write a UTSA CS Division Technical Report". 1995.
- [4] Ralph Griswold. "Icon Benchmarks". The Icon Newsletter. 1989.

- [5] Clinton Jeffery and David Rice. "sesrit.icn". <http://www2.cs.uidaho.edu/~jeffery/courses/game/sesrit.icn>. 1999.
- [6] Brent Fulgham and Isaac Gouy. "The Computer Language Benchmarks Game". <http://benchmarksgame.alioth.debian.org/>.
- [7] Clinton Jeffery. "Merr: an LR Syntax Error Message Tool". <http://unicon.org/merr>.
- [8] anonymous, but owing to Douglas Crawford. "Introducing JSON". json.org.
- [9] Kent Beck. "Simple Smalltalk Testing: With Patterns". Available at <https://web.archive.org/web/20150315073817/http://www.xprogramming.com/testfram.htm> [Verified 13 May 2024].